



Fortify Security Report

8/31/26

1026661

Executive Summary

Issues Overview

On 31-Aug-2026, a source code review was performed over the Credit_Risk_Dossier_SAST code base. 181 files, 0 LOC (Executable) were scanned and reviewed for defects that could lead to potential security vulnerabilities. A total of 88 reviewed findings were uncovered during the analysis.

Issues by Fortify Priority Order

Critical	52
High	23
Medium	9
Low	4

Recommendations and Conclusions

The Issues Category section provides Fortify recommendations for addressing issues at a generic level. The recommendations for specific fixes can be extrapolated from those generic recommendations by the development group.

Project Summary

Code Base Summary

Code location: G:/Jenkins/workspace/IS_BFSI_TCS_Credit_Risk_Dossier_V1_SAST/Credit_Risk_Dossier_SAST/credit-dossier-final-version IP safe_solution
Number of Files: 181
Lines of Code: 0
Build Label: <No Build Label>

Scan Information

Scan time: 04:01
SCA Engine version: 26.2.0.0071
Machine Name: INBBRSEFVM236
Username running scan: INBBRSEFVM236\$

Results Certification

Results Certification Valid

Details:

Results Signature:

SCA Analysis Results has Valid signature

Rules Signature:

rules/custom-rule_PII.xml has Valid signature

Attack Surface

Attack Surface:

Environment Variables:

 null.null.null
 os.null.getenv
 pathlib.Path.expanduser

File System:

 null.null.open
 null.file.readlines

Private Information:

 null.null.null
 null.dict.__getitem__
 hashlib.null.pbkdf2_hmac

System Information:

 null.null.null
 null.null.null
 null.null.__python_get_last_exception

null.null.dir
null.null.fetchDeals
null.null.lambda
null.null.resolve
null.null.useEffect0
null.~JS_Generic.cwd

Web:
null.null.null
null.null.prompt

Filter Set Summary

Current Enabled Filter Set:
Security Auditor View

Filter Set Details:

Folder Filters:
If [fortify priority order] contains critical Then set folder to Critical
If [fortify priority order] contains high Then set folder to High
If [fortify priority order] contains medium Then set folder to Medium
If [fortify priority order] contains low Then set folder to Low
Visibility Filters:

Audit Guide Summary

Code Quality Issues

Hide code quality issues.
Poor code quality leads to unpredictable behavior, poor usability, and an opportunity for an attacker to stress the system in unexpected ways. AuditGuide can hide code quality issues if you are not concerned with them.
Enable if you are not concerned with code quality issues.

Filters:
If kingdom contains code quality Then hide issueSerialization Issues

Hide serialization issues.
Incorrect serialization and de-serialization can lead to unpredictable behavior and cause denial of service attacks. AuditGuide can hide serialization issues if you are not concerned with them.
Enable if you are not concerned with serialization issues.

Filters:
If taint contains serialized Then hide issue
If category is j2ee bad practices: non-serializable object stored in session Then hide issue
If category is incorrect definition of serializable class Then hide issue
If category is serializable class with no version id Then hide issueDenial of Service Attacks

Hide issues that are not directly security-related.
Certain issues may lead to program crashes, but are unlikely to result in malicious code execution. AuditGuide can hide issues that

are based on null pointer dereferences, uninitialized variables, double free bugs, and more.

Enable if you are more concerned about security than reliability.

Filters:

If category is use after free Then hide issue

If category is double free Then hide issue

If category is null dereference Then hide issue

If category is uninitialized variable Then hide issue

If category is missing check against null Then hide issue

Hide poor programming style issues.

Poor programming style usually results in confusing and buggy code. AuditGuide can hide poor programming style issues if you are not concerned with them.

Enable if you are not concerned with poor programming style issues.

Filters:

If category contains poor style Then hide issue

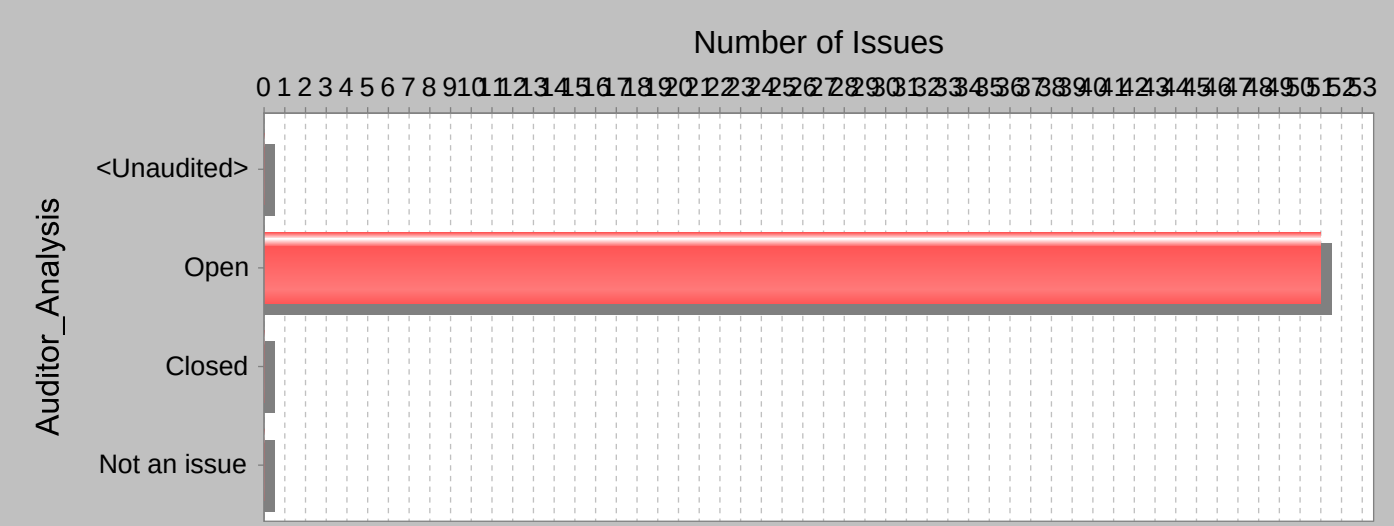
Results Outline

Overall number of results

The scan found 88 issues.

Vulnerability Examples by Category

Category: Privacy Violation (51 Issues)



Abstract:

The file PasswordPolicyChecklist.tsx mishandles confidential information on line 67, which can compromise user privacy and is often illegal.

Explanation:

Privacy violations occur when:

- 1. Private user information enters the program.
- 2. The data is written to an external location, such as the console, file system, or network.

Example 1: The following code stores user's plain text password to the local storage.

```
localStorage.setItem('password', password);
```

Although many developers treat the local storage as a safe location for data, it should not be trusted implicitly, particularly when privacy is a concern.

Private data can enter a program in a variety of ways:

- Directly from the user in the form of a password or personal information
- Accessed from a database or other data store by the application
- Indirectly from a partner or other third party

Sometimes data that is not labeled as private can have a privacy implication in a different context. For example, student identification numbers are usually not considered private because there is no explicit and publicly-available mapping to an individual student's personal information. However, if a school generates identification numbers based on student social security numbers, then the identification numbers should be considered private.

Security and privacy concerns often seem to compete with each other. From a security perspective, you should record all important operations so that any anomalous activity can later be identified. However, when private data is involved, this practice can create risk.

Although there are many ways in which private data can be handled unsafely, a common risk stems from misplaced trust. Programmers often trust the operating environment in which a program runs, and therefore believe that it is acceptable to store private information on the file system, in the registry, or in other locally-controlled resources. However, even if access to certain resources is restricted, this does not guarantee that the individuals who do have access can be trusted. For example, in 2004, an unscrupulous employee at AOL sold approximately 92 million private customer email addresses to a spammer marketing an offshore gambling web site [1].

In response to such high-profile exploits, the collection and management of private data is becoming increasingly regulated. Depending on its location, the type of business it conducts, and the nature of any private data it handles, an organization may be required to comply with one or more of the following federal and state regulations:

- Safe Harbor Privacy Framework [3]
- Gramm-Leach Bliley Act (GLBA) [4]
- Health Insurance Portability and Accountability Act (HIPAA) [5]
- California SB-1386 [6]

Despite these regulations, privacy violations continue to occur with alarming frequency.

Recommendations:

When security and privacy demands clash, privacy should usually be given the higher priority. To accomplish this and still maintain required security information, cleanse any private information before it exits the program.

To enforce good privacy management, develop and strictly adhere to internal privacy guidelines. The guidelines should specifically describe how an application should handle private data. If your organization is regulated by federal or state law, ensure that your privacy guidelines are sufficiently strenuous to meet the legal requirements. Even if your organization is not regulated, you must protect private information or risk losing customer confidence.

The best policy with respect to private data is to minimize its exposure. Applications, processes, and employees should not be granted access to any private data unless the access is required for the tasks that they are to perform. Just as the principle of least privilege dictates that no operation should be performed with more than the necessary privileges, access to private data should be restricted to the smallest possible group.

Tips:

1. As part of any thorough audit for privacy violations, ensure that custom rules are written to identify all sources of private or otherwise sensitive information entering the program. Most sources of private data cannot be identified automatically. Without custom rules, your check for privacy violations is likely to be substantially incomplete.

frontend/src/components/PasswordPolicyChecklist.tsx, line 67 (Privacy Violation)			
Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file PasswordPolicyChecklist.tsx mishandles confidential information on line 67, which can compromise user privacy and is often illegal.		
Source:	PasswordPolicyChecklist.tsx:55 Read passwordPolicyChecks()		
53	userId?: string;		
54	}) {		
55	const checks = passwordPolicyChecks(password, policy, userId);		
56	const meetsPolicy = checks.every((check) => check.met);		
57	return (		
Sink:	PasswordPolicyChecklist.tsx:67 Assignment to textContent()		
65	)}		
66	</div>		
67	<ul className="grid gap-1 text-xs sm:grid-cols-2">		
68	{checks.map((check) => (		
69	<li		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 172 (Privacy Violation)			
Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.		
Source:	login.tsx:21 Read resetPassword()		
19			
20	function LoginPage() {		
21	const { login, register, resetPassword, getResetQuestions } = useAuth();		
22	const navigate = useNavigate();		
23	const [mode, setMode] = useState<"login" "register" "reset">("login");		
Sink:	login.tsx:172 Assignment to textContent()		
170			
171	return (		
172	<main className="relative flex min-h-screen items-center justify-center overflow-hidden bg-slate-950 px-4 py-10">		
173	<div className="absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />		

```

174      <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.

Source: login.tsx:242 Read password()

```

240      }
241      type={showPassword ? "text" : "password"}
242      value={password}
243      onChange={(event) => setPassword(event.target.value)}
244      onCopy={(event) => event.preventDefault()}

```

Sink: login.tsx:172 Assignment to textContent()

```

170
171      return (
172      <main className="relative flex min-h-screen items-center justify-center overflow-
hidden bg-slate-950 px-4 py-10">
173      <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-
gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />
174      <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.

Source: login.tsx:273 Read configuration.password_policy()

```

271      autoComplete="new-password"
272      required
273      minLength={configuration?.password_policy.min_length}
274      maxLength={configuration?.password_policy.max_length ?? 1024}
275      type={showConfirmPassword ? "text" : "password"}

```

Sink: login.tsx:172 Assignment to textContent()

```

170
171      return (
172      <main className="relative flex min-h-screen items-center justify-center overflow-
hidden bg-slate-950 px-4 py-10">
173      <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-
gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />
174      <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">

```

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.

Source: profile.tsx:163 Read setCurrentPassword()

```

161      </div>
162      {[
163      ["Current password", currentPassword, setCurrentPassword],
164      ["New password", newPassword, setNewPassword],
165      ["Confirm new password", confirmPassword, setConfirmPassword],

```

Sink: profile.tsx:134 Assignment to textContent()
 132
 133 return (
 134 <main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">
 135 <div className="doc-card overflow-hidden">
 136 <div className="flex items-center gap-4 border-b bg-primary/5 p-6">

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
-------------------	----------	--------	----------

Kingdom:	Security Features
----------	-------------------

Abstract: The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.

Source: profile.tsx:212 Read newPassword()
 210 {configuration && (
 211 <PasswordPolicyChecklist
 212 password=*****
 213 policy={configuration.password_policy}
 214 userId={user?.user_id}

Sink: profile.tsx:134 Assignment to textContent()
 132
 133 return (
 134 <main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">
 135 <div className="doc-card overflow-hidden">
 136 <div className="flex items-center gap-4 border-b bg-primary/5 p-6">

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 249 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
-------------------	----------	--------	----------

Kingdom:	Security Features
----------	-------------------

Abstract: The file profile.tsx mishandles confidential information on line 249, which can compromise user privacy and is often illegal.

Source: profile.tsx:255 Read securityPassword()
 253 type="password"
 254 autoComplete="current-password"
 255 value={securityPassword}
 256 onChange={(event) => setSecurityPassword(event.target.value)}
 257 className="h-10 w-full rounded-md border border-input bg-background px-3
 font-normal outline-none focus:ring-2 focus:ring-ring"

Sink: profile.tsx:249 Assignment to textContent()
 247 allowCustomQuestions={configuration?.allow_custom_security_questions ??
 false}
 248 />
 249 <label className="block space-y-1.5 text-sm font-medium">
 250 Current password
 251 <input

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
-------------------	----------	--------	----------

Kingdom:	Security Features
----------	-------------------

Abstract: The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.

Source: profile.tsx:255 Read securityPassword()
 253 type="password"
 254 autoComplete="current-password"
 255 value={securityPassword}
 256 onChange={(event) => setSecurityPassword(event.target.value)}

```

257             className="h-10 w-full rounded-md border border-input bg-background px-3
font-normal outline-none focus:ring-2 focus:ring-ring"
Sink:      profile.tsx:134 Assignment to textContent()
132
133         return (
134             <main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">
135                 <div className="doc-card overflow-hidden">
136                     <div className="flex items-center gap-4 border-b bg-primary/5 p-6">
Auditor_Analysis:  Open

```

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.		
Source:	login.tsx:274 Read configuration.password_policy()		
272	required		
273	minLength={configuration?.password_policy.min_length}		
274	maxLength={configuration?.password_policy.max_length ?? 1024}		
275	type={showConfirmPassword ? "text" : "password"}		
276	value={confirmPassword}		
Sink:	login.tsx:172 Assignment to textContent()		
170			
171	return (		
172	<main className="relative flex min-h-screen items-center justify-center overflow-hidden bg-slate-950 px-4 py-10">		
173	<div className="absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />		
174	<div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.		
Source:	login.tsx:251 Read configuration.password_policy()		
249	mode === "login"		
250	? "Enter your password"		
251	: `Minimum \${configuration?.password_policy.min_length ?? "configured"} characters`		
252	}		
253	/>		
Sink:	login.tsx:172 Assignment to textContent()		
170			
171	return (		
172	<main className="relative flex min-h-screen items-center justify-center overflow-hidden bg-slate-950 px-4 py-10">		
173	<div className="absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />		
174	<div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.		

Source: login.tsx:347 Read showPasswordField()
 345 })
 346
 347 {mode !== "login" && showPasswordField && configuration && (
 348 <PasswordPolicyChecklist
 349 password=*****
Sink: login.tsx:172 Assignment to textContent()
 170
 171 return (
 172 <main className="relative flex min-h-screen items-center justify-center overflow-
 hidden bg-slate-950 px-4 py-10">
 173 <div className="absolute inset-0 bg-[radial-
 gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-
 gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />
 174 <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
 border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">
Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.

Source: login.tsx:230 Read showPasswordField()
 228 />>
 229 </label>
 230 {showPasswordField && (
 231 <label className="block space-y-1.5 text-sm font-medium">
 232 {mode === "reset" ? "New password" : "Password"}
Sink: login.tsx:193 Assignment to textContent()
 191 </section>
 192
 193 <section className="p-6 sm:p-10">
 194 <div className="mb-7 md:hidden">
 195 <div className="flex items-center gap-2 font-semibold text-primary">

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 167 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file profile.tsx mishandles confidential information on line 167, which can compromise user privacy and is often illegal.

Source: profile.tsx:164 Read setNewPassword()
 162 [
 163 ["Current password", currentPassword, setCurrentPassword],
 164 ["New password", newPassword, setNewPassword],
 165 ["Confirm new password", confirmPassword, setConfirmPassword],
 166].map(([label, value, setter]) => (
Sink: profile.tsx:167 Assignment to textContent()
 165 ["Confirm new password", confirmPassword, setConfirmPassword],
 166].map(([label, value, setter]) => (
 167 <label key={label as string} className="block space-y-1.5 text-sm font-
 medium">
 168 {label as string}
 169 <div className="relative">

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 167 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract:	The file profile.tsx mishandles confidential information on line 167, which can compromise user privacy and is often illegal.
Source:	profile.tsx:164 Read newPassword() <pre> 162 { 163 ["Current password", currentPassword, setCurrentPassword], 164 ["New password", newPassword, setNewPassword], 165 ["Confirm new password", confirmPassword, setConfirmPassword], 166].map(([label, value, setter]) => (</pre>
Sink:	profile.tsx:167 Assignment to textContent() <pre> 165 ["Confirm new password", confirmPassword, setConfirmPassword], 166].map(([label, value, setter]) => (167 <label key={label as string} className="block space-y-1.5 text-sm font- medium"> 168 {label as string} 169 <div className="relative"> </pre>
Auditor_Analysis:	Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.		
Source:	login.tsx:349 Read password() <pre> 347 {mode !== "login" && showPasswordField && configuration && (348 <PasswordPolicyChecklist 349 password=***** 350 policy={configuration.password_policy} 351 userId={userId} </pre>		
Sink:	login.tsx:193 Assignment to textContent() <pre> 191 </section> 192 193 <section className="p-6 sm:p-10"> 194 <div className="mb-7 md:hidden"> 195 <div className="flex items-center gap-2 font-semibold text-primary"> </pre>		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.		
Source:	login.tsx:273 Read configuration.password_policy() <pre> 271 autoComplete="new-password" 272 required 273 minLength={configuration?.password_policy.min_length} 274 maxLength={configuration?.password_policy.max_length ?? 1024} 275 type={showConfirmPassword ? "text" : "password"} </pre>		
Sink:	login.tsx:193 Assignment to textContent() <pre> 191 </section> 192 193 <section className="p-6 sm:p-10"> 194 <div className="mb-7 md:hidden"> 195 <div className="flex items-center gap-2 font-semibold text-primary"> </pre>		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract:	The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.
Source:	login.tsx:350 Read configuration.password_policy() 348 <PasswordPolicyChecklist 349 password=***** 350 policy={configuration.password_policy} 351 userId={userId} 352 />
Sink:	login.tsx:172 Assignment to textContent() 170 171 return (172 <main className="relative flex min-h-screen items-center justify-center overflow-hidden bg-slate-950 px-4 py-10"> 173 <div className="absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" /> 174 <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">
Auditor_Analysis:	Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.		
Source:	login.tsx:274 Read configuration.password_policy() 272 required 273 minLength={configuration?.password_policy.min_length} 274 maxLength={configuration?.password_policy.max_length ?? 1024} 275 type={showConfirmPassword ? "text" : "password"} 276 value={confirmPassword}		
Sink:	login.tsx:193 Assignment to textContent() 191 </section> 192 193 <section className="p-6 sm:p-10"> 194 <div className="mb-7 md:hidden"> 195 <div className="flex items-center gap-2 font-semibold text-primary">		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 267 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 267, which can compromise user privacy and is often illegal.		
Source:	login.tsx:274 Read configuration.password_policy() 272 required 273 minLength={configuration?.password_policy.min_length} 274 maxLength={configuration?.password_policy.max_length ?? 1024} 275 type={showConfirmPassword ? "text" : "password"} 276 value={confirmPassword}		
Sink:	login.tsx:267 Assignment to textContent() 265 266 {mode !== "login" && showPasswordField && (267 <label className="block space-y-1.5 text-sm font-medium"> 268 Confirm {mode === "reset" ? "new " : ""}password 269 <div className="relative">		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
--------------------------	----------	--------	----------

Kingdom:	Security Features
Abstract:	The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.
Source:	login.tsx:239 Read configuration.password_policy() 237 minLength={mode === "login" ? 1 : configuration?.password_policy.min_length} 238 maxLength={ 239 mode === "login" ? 1024 : (configuration?.password_policy.max_length ?? 1024) 240 } 241 type={showPassword ? "text" : "password"} Sink: login.tsx:193 Assignment to textContent() 191 </section> 192 193 <section className="p-6 sm:p-10"> 194 <div className="mb-7 md:hidden"> 195 <div className="flex items-center gap-2 font-semibold text-primary">
Auditor_Analysis:	Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.		
Source:	login.tsx:266 Read showPasswordField() 264 }) 265 266 {mode !== "login" && showPasswordField && (267 <label className="block space-y-1.5 text-sm font-medium"> 268 Confirm {mode === "reset" ? "new " : ""}password Sink: login.tsx:193 Assignment to textContent() 191 </section> 192 193 <section className="p-6 sm:p-10"> 194 <div className="mb-7 md:hidden"> 195 <div className="flex items-center gap-2 font-semibold text-primary">		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 267 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 267, which can compromise user privacy and is often illegal.		
Source:	login.tsx:276 Read confirmPassword() 274 maxLength={configuration?.password_policy.max_length ?? 1024} 275 type={showConfirmPassword ? "text" : "password"} 276 value={confirmPassword} 277 onChange={(event) => setConfirmPassword(event.target.value)} 278 onCopy={(event) => event.preventDefault()} Sink: login.tsx:267 Assignment to textContent() 265 266 {mode !== "login" && showPasswordField && (267 <label className="block space-y-1.5 text-sm font-medium"> 268 Confirm {mode === "reset" ? "new " : ""}password 269 <div className="relative">		
Auditor_Analysis:	Open		

frontend/src/routes/profile.tsx, line 167 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
--------------------------	----------	--------	----------

Kingdom:	Security Features
Abstract:	The file profile.tsx mishandles confidential information on line 167, which can compromise user privacy and is often illegal.
Source:	<pre> profile.tsx:165 Read setConfirmPassword() 163 ["Current password", currentPassword, setCurrentPassword], 164 ["New password", newPassword, setNewPassword], 165 ["Confirm new password", confirmPassword, setConfirmPassword], 166].map(([label, value, setter]) => (167 <label key={label as string} className="block space-y-1.5 text-sm font- medium"> </pre>
Sink:	<pre> profile.tsx:167 Assignment to textContent() 165 ["Confirm new password", confirmPassword, setConfirmPassword], 166].map(([label, value, setter]) => (167 <label key={label as string} className="block space-y-1.5 text-sm font- medium"> 168 {label as string} 169 <div className="relative"> </pre>
Auditor_Analysis:	Open

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.		
Source:	<pre> profile.tsx:29 Read changePassword() 27 28 function ProfilePage() { 29 const { user, changePassword, configureSecurityQuestions, logout } = useAuth(); 30 const navigate = useNavigate(); 31 const [currentPassword, setCurrentPassword] = useState(""); </pre>		
Sink:	<pre> profile.tsx:134 Assignment to textContent() 132 133 return (134 <main className="mx-auto max-w-3xl px-4 py-8 sm:px-6"> 135 <div className="doc-card overflow-hidden"> 136 <div className="flex items-center gap-4 border-b bg-primary/5 p-6"> </pre>		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.		
Source:	<pre> login.tsx:237 Read configuration.password_policy() 235 autoComplete={mode === "login" ? "current-password" : "new- password"} 236 required 237 minLength={mode === "login" ? 1 : configuration?.password_policy.min_length} 238 maxLength={ 239 mode === "login" ? 1024 : (configuration?.password_policy.max_length ?? 1024) </pre>		
Sink:	<pre> login.tsx:172 Assignment to textContent() 170 171 return (172 <main className="relative flex min-h-screen items-center justify-center overflow- hidden bg-slate-950 px-4 py-10"> 173 <div className="absolute inset-0 bg-[radial- gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial- gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" /> 174 <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]"> </pre>		

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 167 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file profile.tsx mishandles confidential information on line 167, which can compromise user privacy and is often illegal.

Source: profile.tsx:163 Read currentPassword()

```

161         </div>
162         [
163             ["Current password", currentPassword, setCurrentPassword],
164             ["New password", newPassword, setNewPassword],
165             ["Confirm new password", confirmPassword, setConfirmPassword],
Sink: profile.tsx:167 Assignment to textContent()
165             ["Confirm new password", confirmPassword, setConfirmPassword],
166             ].map(([label, value, setter]) => (
167                 <label key={label as string} className="block space-y-1.5 text-sm font-
medium">
168                     {label as string}
169                 <div className="relative">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.

Source: login.tsx:348 Read PasswordPolicyChecklist.prototype()

```

346
347         {mode !== "login" && showPasswordField && configuration && (
348             <PasswordPolicyChecklist
349                 password=*****
350                 policy={configuration.password_policy}
Sink: login.tsx:193 Assignment to textContent()
191         </section>
192
193         <section className="p-6 sm:p-10">
194             <div className="mb-7 md:hidden">
195                 <div className="flex items-center gap-2 font-semibold text-primary">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.

Source: login.tsx:348 Read PasswordPolicyChecklist.prototype()

```

346
347         {mode !== "login" && showPasswordField && configuration && (
348             <PasswordPolicyChecklist
349                 password=*****
350                 policy={configuration.password_policy}
Sink: login.tsx:172 Assignment to textContent()
170
171         return (
172             <main className="relative flex min-h-screen items-center justify-center overflow-
hidden bg-slate-950 px-4 py-10">

```

```
173         <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-
gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />
174         <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">
```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 231 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 231, which can compromise user privacy and is often illegal.		
Source:	login.tsx:239 Read configuration.password_policy()		
237	minLength={mode === "login" ? 1 :		
	configuration?.password_policy.min_length}		
238	maxLength={		
239	mode === "login" ? 1024 :		
	(configuration?.password_policy.max_length ?? 1024)		
240	}		
241	type={showPassword ? "text" : "password"}		
Sink:	login.tsx:231 Assignment to textContent()		
229	</label>		
230	{showPasswordField && (		
231	<label className="block space-y-1.5 text-sm font-medium">		
232	{mode === "reset" ? "New password" : "Password"}		
233	<div className="relative">		

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 167 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file profile.tsx mishandles confidential information on line 167, which can compromise user privacy and is often illegal.		
Source:	profile.tsx:165 Read confirmPassword()		
163	["Current password", currentPassword, setCurrentPassword],		
164	["New password", newPassword, setNewPassword],		
165	["Confirm new password", confirmPassword, setConfirmPassword],		
166	].map(([label, value, setter]) => (		
167	<label key={label as string} className="block space-y-1.5 text-sm font-		
	medium">		
Sink:	profile.tsx:167 Assignment to textContent()		
165	["Confirm new password", confirmPassword, setConfirmPassword],		
166	].map(([label, value, setter]) => (		
167	<label key={label as string} className="block space-y-1.5 text-sm font-		
	medium">		
168	{label as string}		
169	<div className="relative">		

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 231 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 231, which can compromise user privacy and is often illegal.		
Source:	login.tsx:251 Read configuration.password_policy()		
249	mode === "login"		
250	? "Enter your password"		
251	: `Minimum \${configuration?.password_policy.min_length ??		
	"configured"} characters`		
252	}		
253	/>		

Sink: login.tsx:231 Assignment to textContent()
 229 </label>
 230 {showPasswordField && (
 231 <label className="block space-y-1.5 text-sm font-medium">
 232 {mode === "reset" ? "New password" : "Password"}
 233 <div className="relative">

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.

Source: profile.tsx:213 Read configuration.password_policy()
 211 <PasswordPolicyChecklist
 212 password=*****
 213 policy={configuration.password_policy}
 214 userId={user?.user_id}
 215 />

Sink: profile.tsx:134 Assignment to textContent()
 132
 133 return (
 134 <main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">
 135 <div className="doc-card overflow-hidden">
 136 <div className="flex items-center gap-4 border-b bg-primary/5 p-6">

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.

Source: login.tsx:237 Read configuration.password_policy()
 235 autoComplete={mode === "login" ? "current-password" : "new-
 password"}
 236 required
 237 minLength={mode === "login" ? 1 :
 configuration?.password_policy.min_length}
 238 maxLength={
 239 mode === "login" ? 1024 :
 (configuration?.password_policy.max_length ?? 1024)}

Sink: login.tsx:193 Assignment to textContent()
 191 </section>
 192
 193 <section className="p-6 sm:p-10">
 194 <div className="mb-7 md:hidden">
 195 <div className="flex items-center gap-2 font-semibold text-primary">

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 267 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file login.tsx mishandles confidential information on line 267, which can compromise user privacy and is often illegal.

Source: login.tsx:273 Read configuration.password_policy()
 271 autoComplete="new-password"
 272 required
 273 minLength={configuration?.password_policy.min_length}
 274 maxLength={configuration?.password_policy.max_length ?? 1024}

```
275                                     type={showConfirmPassword ? "text" : "password"}
```

Sink: login.tsx:267 Assignment to textContent()

```
265
266                                     {mode !== "login" && showPasswordField && (
267                                     <label className="block space-y-1.5 text-sm font-medium">
268                                     Confirm {mode === "reset" ? "new " : ""}password
269                                     <div className="relative">
```

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
--------------------------	----------	---------------	----------

Kingdom:	Security Features
-----------------	-------------------

Abstract: The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.

Source: profile.tsx:211 Read PasswordPolicyChecklist.prototype()

```
209                                     )})
210                                     {configuration && (
211                                     <PasswordPolicyChecklist
212                                     password=*****
213                                     policy={configuration.password_policy}
```

Sink: profile.tsx:134 Assignment to textContent()

```
132
133                                     return (
134                                     <main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">
135                                     <div className="doc-card overflow-hidden">
136                                     <div className="flex items-center gap-4 border-b bg-primary/5 p-6">
```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
--------------------------	----------	---------------	----------

Kingdom:	Security Features
-----------------	-------------------

Abstract: The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.

Source: login.tsx:266 Read showPasswordField()

```
264                                     )}
265
266                                     {mode !== "login" && showPasswordField && (
267                                     <label className="block space-y-1.5 text-sm font-medium">
268                                     Confirm {mode === "reset" ? "new " : ""}password
```

Sink: login.tsx:172 Assignment to textContent()

```
170
171                                     return (
172                                     <main className="relative flex min-h-screen items-center justify-center overflow-
hidden bg-slate-950 px-4 py-10">
173                                     <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-
gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />
174                                     <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">
```

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
--------------------------	----------	---------------	----------

Kingdom:	Security Features
-----------------	-------------------

Abstract: The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.

Source: profile.tsx:174 Read configuration.password_policy()

```
172                                     required
173                                     minLength={
```

```

174             label === "Current password" ? 1 :
configuration?.password_policy.min_length
175         }
176         maxLength={
Sink:      profile.tsx:134 Assignment to textContent()
132
133         return (
134             <main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">
135                 <div className="doc-card overflow-hidden">
136                     <div className="flex items-center gap-4 border-b bg-primary/5 p-6">

```

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 167 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file profile.tsx mishandles confidential information on line 167, which can compromise user privacy and is often illegal.

Source: profile.tsx:163 Read setCurrentPassword()

```

161         </div>
162         {[
163             ["Current password", currentPassword, setCurrentPassword],
164             ["New password", newPassword, setNewPassword],
165             ["Confirm new password", confirmPassword, setConfirmPassword],
Sink:      profile.tsx:167 Assignment to textContent()
165             ["Confirm new password", confirmPassword, setConfirmPassword],
166             ].map(([label, value, setter]) => (
167                 <label key={label as string} className="block space-y-1.5 text-sm font-
medium">
168                     {label as string}
169                 <div className="relative">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.

Source: login.tsx:276 Read confirmPassword()

```

274             maxLength={configuration?.password_policy.max_length ?? 1024}
275             type={showConfirmPassword ? "text" : "password"}
276             value={confirmPassword}
277             onChange={(event) => setConfirmPassword(event.target.value)}
278             onCopy={(event) => event.preventDefault()}

```

Sink: login.tsx:193 Assignment to textContent()

```

191         </section>
192
193         <section className="p-6 sm:p-10">
194             <div className="mb-7 md:hidden">
195                 <div className="flex items-center gap-2 font-semibold text-primary">

```

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority: Critical Folder Critical

Kingdom: Security Features

Abstract: The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.

Source: profile.tsx:179 Read configuration.password_policy()

```

177             label === "Current password"
178             ? 1024

```

```

179                                     : (configuration?.password_policy.max_length ?? 1024)
180                                     }
181                                     value={value as string}
Sink:      profile.tsx:134 Assignment to textContent()
132
133         return (
134             <main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">
135                 <div className="doc-card overflow-hidden">
136                     <div className="flex items-center gap-4 border-b bg-primary/5 p-6">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 231 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file login.tsx mishandles confidential information on line 231, which can compromise user privacy and is often illegal.

Source: login.tsx:237 Read configuration.password_policy()

```

235         autoComplete={mode === "login" ? "current-password" : "new-
password"}
236         required
237         minLength={mode === "login" ? 1 :
configuration?.password_policy.min_length}
238         maxLength={
239             mode === "login" ? 1024 :
(configuration?.password_policy.max_length ?? 1024)

```

Sink: login.tsx:231 Assignment to textContent()

```

229         </label>
230         {showPasswordField && (
231             <label className="block space-y-1.5 text-sm font-medium">
232                 {mode === "reset" ? "New password" : "Password"}
233                 <div className="relative">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.

Source: login.tsx:242 Read password()

```

240         }
241         type={showPassword ? "text" : "password"}
242         value={password}
243         onChange={(event) => setPassword(event.target.value)}
244         onCopy={(event) => event.preventDefault()}

```

Sink: login.tsx:193 Assignment to textContent()

```

191         </section>
192
193         <section className="p-6 sm:p-10">
194             <div className="mb-7 md:hidden">
195                 <div className="flex items-center gap-2 font-semibold text-primary">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.

Source: login.tsx:350 Read configuration.password_policy()

```

348         <PasswordPolicyChecklist

```

```

349         password=*****
350         policy={configuration.password_policy}
351         userId={userId}
352     />
Sink:      login.tsx:193 Assignment to textContent()
191         </section>
192
193         <section className="p-6 sm:p-10">
194             <div className="mb-7 md:hidden">
195                 <div className="flex items-center gap-2 font-semibold text-primary">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.

Source: login.tsx:239 Read configuration.password_policy()

```

237         minLength={mode === "login" ? 1 :
configuration?.password_policy.min_length}
238         maxLength={
239             mode === "login" ? 1024 :
(configuration?.password_policy.max_length ?? 1024)
240         }
241         type={showPassword ? "text" : "password"}

```

Sink: login.tsx:172 Assignment to textContent()

```

170
171     return (
172         <main className="relative flex min-h-screen items-center justify-center overflow-
hidden bg-slate-950 px-4 py-10">
173         <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-
gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />
174         <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		

Abstract: The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.

Source: login.tsx:349 Read password()

```

347         {mode !== "login" && showPasswordField && configuration && (
348             <PasswordPolicyChecklist
349                 password=*****
350                 policy={configuration.password_policy}
351                 userId={userId}

```

Sink: login.tsx:172 Assignment to textContent()

```

170
171     return (
172         <main className="relative flex min-h-screen items-center justify-center overflow-
hidden bg-slate-950 px-4 py-10">
173         <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-
gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />
174         <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
-------------------	----------	--------	----------

Kingdom:	Security Features
Abstract:	The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.
Source:	login.tsx:347 Read showPasswordField() 345 } 346 347 {mode !== "login" && showPasswordField && configuration && (348 <PasswordPolicyChecklist 349 password=*****
Sink:	login.tsx:193 Assignment to textContent() 191 </section> 192 193 <section className="p-6 sm:p-10"> 194 <div className="mb-7 md:hidden"> 195 <div className="flex items-center gap-2 font-semibold text-primary">
Auditor_Analysis:	Open

frontend/src/routes/login.tsx, line 231 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 231, which can compromise user privacy and is often illegal.		
Source:	login.tsx:242 Read password() 240 } 241 type={showPassword ? "text" : "password"} 242 value={password} 243 onChange={(event) => setPassword(event.target.value)} 244 onCopy={(event) => event.preventDefault()}		
Sink:	login.tsx:231 Assignment to textContent() 229 </label> 230 {showPasswordField && (231 <label className="block space-y-1.5 text-sm font-medium"> 232 {mode === "reset" ? "New password" : "Password"} 233 <div className="relative">		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.		
Source:	login.tsx:230 Read showPasswordField() 228 /> 229 </label> 230 {showPasswordField && (231 <label className="block space-y-1.5 text-sm font-medium"> 232 {mode === "reset" ? "New password" : "Password"} 233 <div className="relative">		
Sink:	login.tsx:172 Assignment to textContent() 170 171 return (172 <main className="relative flex min-h-screen items-center justify-center overflow-hidden bg-slate-950 px-4 py-10"> 173 <div className="absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" /> 174 <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 172 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 172, which can compromise user privacy and is often illegal.		
Source:	login.tsx:276 Read confirmPassword()		
274	maxLength={configuration?.password_policy.max_length ?? 1024}		
275	type={showConfirmPassword ? "text" : "password"}		
276	value={confirmPassword}		
277	onChange={(event) => setConfirmPassword(event.target.value)}		
278	onCopy={(event) => event.preventDefault()}		
Sink:	login.tsx:172 Assignment to textContent()		
170			
171	return (		
172	<main className="relative flex min-h-screen items-center justify-center overflow-hidden bg-slate-950 px-4 py-10">		
173	<div className="absolute inset-0 bg-[radial-gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />		
174	<div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">		
Auditor_Analysis:	Open		

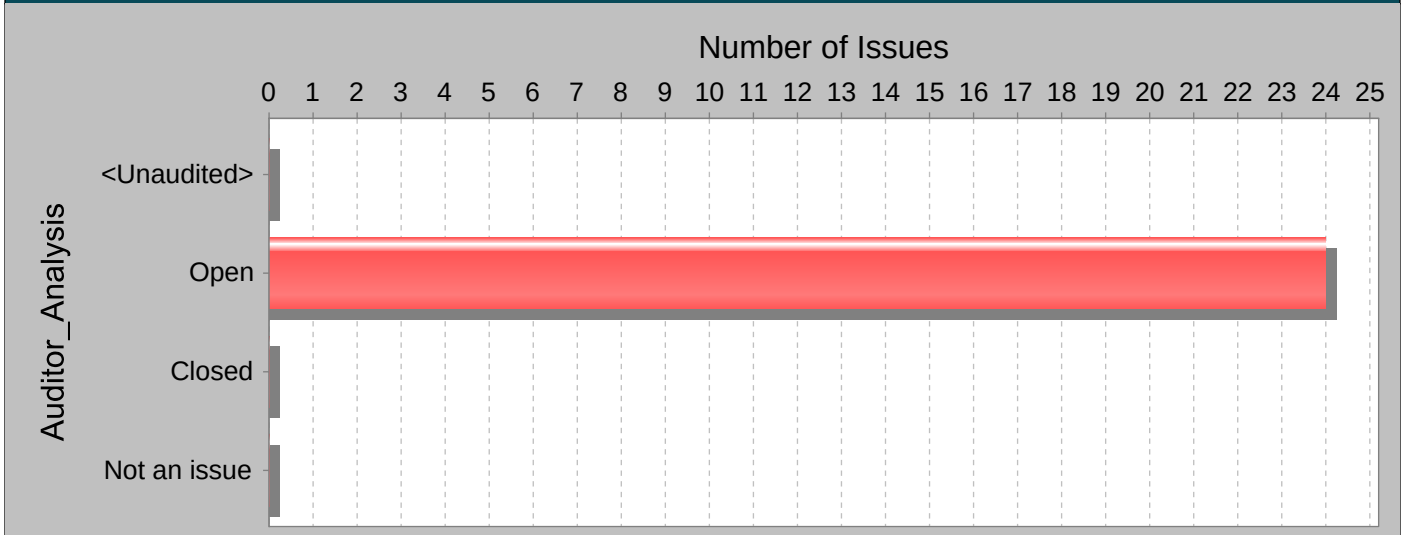
frontend/src/routes/login.tsx, line 193 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file login.tsx mishandles confidential information on line 193, which can compromise user privacy and is often illegal.		
Source:	login.tsx:251 Read configuration.password_policy()		
249	mode === "login"		
250	? "Enter your password"		
251	: `Minimum \${configuration?.password_policy.min_length ?? "configured"} characters`		
252	}		
253	/>		
Sink:	login.tsx:193 Assignment to textContent()		
191	</section>		
192			
193	<section className="p-6 sm:p-10">		
194	<div className="mb-7 md:hidden">		
195	<div className="flex items-center gap-2 font-semibold text-primary">		
Auditor_Analysis:	Open		

frontend/src/routes/profile.tsx, line 134 (Privacy Violation)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	The file profile.tsx mishandles confidential information on line 134, which can compromise user privacy and is often illegal.		
Source:	profile.tsx:163 Read currentPassword()		
161	</div>		
162	{[		
163	["Current password", currentPassword, setCurrentPassword],		
164	["New password", newPassword, setNewPassword],		
165	["Confirm new password", confirmPassword, setConfirmPassword],		
Sink:	profile.tsx:134 Assignment to textContent()		
132			
133	return (		
134	<main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">		
135	<div className="doc-card overflow-hidden">		
136	<div className="flex items-center gap-4 border-b bg-primary/5 p-6">		
Auditor_Analysis:	Open		

Category: Password Management: Hardcoded Password (24 Issues)



Abstract:

Hardcoded passwords can compromise system security in a way that is difficult to remedy.

Explanation:

Never hardcode passwords. Not only does it expose the password to all of the project's developers, it also makes fixing the problem extremely difficult. After the code is in production, a program patch is probably the only way to change the password. If the account protected by the password is compromised, the organization must choose between security and system availability.

Example 1: The following URL uses a hardcoded password:

```
...
https://user:secretpassword@example.com
...
```

Example 2: The following ODBC connection string uses a hardcoded password:

```
...
server=Server;database=Database;UID=UserName;PWD=Password;Encrypt=yes;
...
```

Recommendations:

Example 3: The following ODBC connection string addresses the hardcoded password issue in Example 2 by implementing Integrated Windows Authentication:

```
...
server=Server;database=Database;Trusted_Connection=yes;Encrypt=yes;
...
```

Never use hardcoded passwords. Always obfuscate and manage passwords in an external source. Storing passwords in plain text anywhere on the system enables anyone with sufficient permissions to read and potentially misuse the password.

backend/.env.example, line 15 (Password Management: Hardcoded Password)

Fortify Priority:	Critical	Folder	Critical
Kingdom:	Security Features		
Abstract:	Hardcoded passwords can compromise system security in a way that is difficult to remedy.		
Sink:	.env.example:15		
13	# Database Configuration		
14	# Local PostgreSQL database used only by the backend application:		
15	DATABASE_URL=postgresql+psycopg://postgres:your_password@127.0.0.1:5432/credit_dossier_backend		
16	DB_LOCK_TIMEOUT_SECONDS=5		
Auditor_Analysis:	Open		

backend/tests/test_auth_recovery.py, line 153 (Password Management: Hardcoded Password)

Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_auth_recovery.py:153 FunctionCall: __setitem__()		
151	json={		
152	"current_password": "WrongCurrentPassword1!",		
153	"new_password": "FinalPassword3!",		
154	"confirm_password": "FinalPassword3!",		
155	},		
Auditor_Analysis:	Open		
backend/tests/test_auth_recovery.py, line 479 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_auth_recovery.py:479 FunctionCall: __setitem__()		
477	try:		
478	first_login = first_client.post(		
479	"/api/auth/login", json={"user_id": user_id, "password": password}		
480	)		
481	assert first_login.status_code == 200, first_login.text		
Auditor_Analysis:	Open		
backend/tests/test_auth_recovery.py, line 303 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_auth_recovery.py:303 FunctionCall: __setitem__()		
301	analyst_login = client.post(		
302	"/api/auth/login",		
303	json={"user_id": "policy-analyst", "password": "AnalystPassword1!"},		
304	)		
305	analyst_cookie = analyst_login.headers["set-cookie"]		
Auditor_Analysis:	Open		
backend/tests/test_security_controls.py, line 50 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_security_controls.py:50 FunctionCall: __setitem__()		
48	rejected = client.request("GET", "/data", content=b'{"secret":"value"}')		
49	rejected_query = client.get("/data?secret=value")		
50	accepted = client.post("/data", json={"secret": "value"})		
51	self.assertEqual(rejected.status_code, 405)		
52	self.assertEqual(rejected_query.status_code, 405)		
Auditor_Analysis:	Open		
backend/tests/test_auth_recovery.py, line 248 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		

Sink: test_auth_recovery.py:248 FunctionCall: __setitem__()
 246 admin_login = client.post(
 247 "/api/auth/login",
 248 json={"user_id": administrator_id, "password": "AdminPassword1!"},
 249)
 250 assert admin_login.status_code == 200, admin_login.text

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 485 (Password Management: Hardcoded Password)

Fortify Priority:	High	Folder	High
Kingdom:	Security Features		

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:485 FunctionCall: __setitem__()
 483
 484 second_login = second_client.post(
 485 "/api/auth/login", json={"user_id": user_id, "password": password}
 486)
 487 assert second_login.status_code == 200, second_login.text

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 161 (Password Management: Hardcoded Password)

Fortify Priority:	High	Folder	High
Kingdom:	Security Features		

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:161 FunctionCall: __setitem__()
 159 "/api/auth/change-password",
 160 json={
 161 "current_password": "DifferentPassword2!",
 162 "new_password": "FinalPassword3!",
 163 "confirm_password": "FinalPassword3!",

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 76 (Password Management: Hardcoded Password)

Fortify Priority:	High	Folder	High
Kingdom:	Security Features		

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:76 FunctionCall: __setitem__()
 74 json={
 75 "user_id": "recovery-test",
 76 "password": "StrongPassword1!",
 77 "confirm_password": "StrongPassword1!",
 78 "role": "credit_analyst",

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 152 (Password Management: Hardcoded Password)

Fortify Priority:	High	Folder	High
Kingdom:	Security Features		

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:152 FunctionCall: __setitem__()
 150 "/api/auth/change-password",
 151 json={
 152 "current_password": "WrongCurrentPassword1!",
 153 "new_password": "FinalPassword3!",

154 "confirm_password": "FinalPassword3!",

Auditor_Analysis: Open

backend/tests/test_security_controls.py, line 48 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom: Security Features

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_security_controls.py:48 StringLiteral()

```
46
47         client = TestClient(app)
48         rejected = client.request("GET", "/data", content=b'{"secret":"value"}')
49         rejected_query = client.get("/data?secret=value")
50         accepted = client.post("/data", json={"secret": "value"})
```

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 125 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom: Security Features

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:125 FunctionCall: __setitem__()

```
123         assert client.post(
124             "/api/auth/login",
125             json={"user_id": "recovery-test", "password": "StrongPassword1!"},
126             ).status_code == 401
127         assert client.post(
```

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 314 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom: Security Features

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:314 FunctionCall: __setitem__()

```
312         admin_login = client.post(
313             "/api/auth/login",
314             json={"user_id": "policy-admin", "password": "AdminPassword1!"},
315             )
316         admin_cookie = admin_login.headers["set-cookie"]
```

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 129 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom: Security Features

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:129 FunctionCall: __setitem__()

```
127         assert client.post(
128             "/api/auth/login",
129             json={"user_id": "recovery-test", "password": "DifferentPassword2!"},
130             ).status_code == 200
```

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 162 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_auth_recovery.py:162 FunctionCall: __setitem__()		
160	json={		
161	"current_password": "DifferentPassword2!",		
162	"new_password": "FinalPassword3!",		
163	"confirm_password": "FinalPassword3!",		
164	},		
Auditor_Analysis:	Open		
backend/tests/test_auth_recovery.py, line 261 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_auth_recovery.py:261 FunctionCall: __setitem__()		
259			
260	approved_login = client.post(		
261	"/api/auth/login", json={"user_id": user_id, "password": password}		
262	)		
263	assert approved_login.status_code == 200, approved_login.text		
Auditor_Analysis:	Open		
backend/tests/test_auth_recovery.py, line 199 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_auth_recovery.py:199 VariableAccess: password()		
197	user_id = "pending-approval-test"		
198	administrator_id = "approval-admin-test"		
199	password =*****		
200	questions = [		
201	{"question": question, "answer": f"Answer {index}"}		
Auditor_Analysis:	Open		
backend/tests/test_auth_recovery.py, line 461 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_auth_recovery.py:461 VariableAccess: password()		
459	auth.PBKDF2_ITERATIONS = 1_000		
460	user_id = "single-session-test"		
461	password =*****		
462	db = TestingSession()		
463	try:		
Auditor_Analysis:	Open		
backend/tests/test_auth_recovery.py, line 118 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords could compromise system security in a way that is not easy to remedy.		
Sink:	test_auth_recovery.py:118 FunctionCall: setitem ()		

```

116         "user_id": "recovery-test",
117         "security_questions": responses,
118         "new_password": "DifferentPassword2!",
119         "confirm_password": "DifferentPassword2!",
120     },

```

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 139 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom: Security Features

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:139 FunctionCall: __setitem__()

```

137         "/api/auth/security-questions",
138         json={
139             "current_password": "DifferentPassword2!",
140             "security_questions": replacement,
141         },

```

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 241 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom: Security Features

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:241 FunctionCall: __setitem__()

```

239
240         pending_login = client.post(
241             "/api/auth/login", json={"user_id": user_id, "password": "wrong"}
242         )
243         assert pending_login.status_code == 403

```

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 107 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom: Security Features

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

Sink: test_auth_recovery.py:107 FunctionCall: __setitem__()

```

105         "user_id": "recovery-test",
106         "security_questions": incorrect,
107         "new_password": "DifferentPassword2!",
108         "confirm_password": "DifferentPassword2!",
109     },

```

Auditor_Analysis: Open

backend/tests/test_auth_recovery.py, line 223 (Password Management: Hardcoded Password)

Fortify Priority: High Folder High

Kingdom: Security Features

Abstract: Hardcoded passwords could compromise system security in a way that is not easy to remedy.

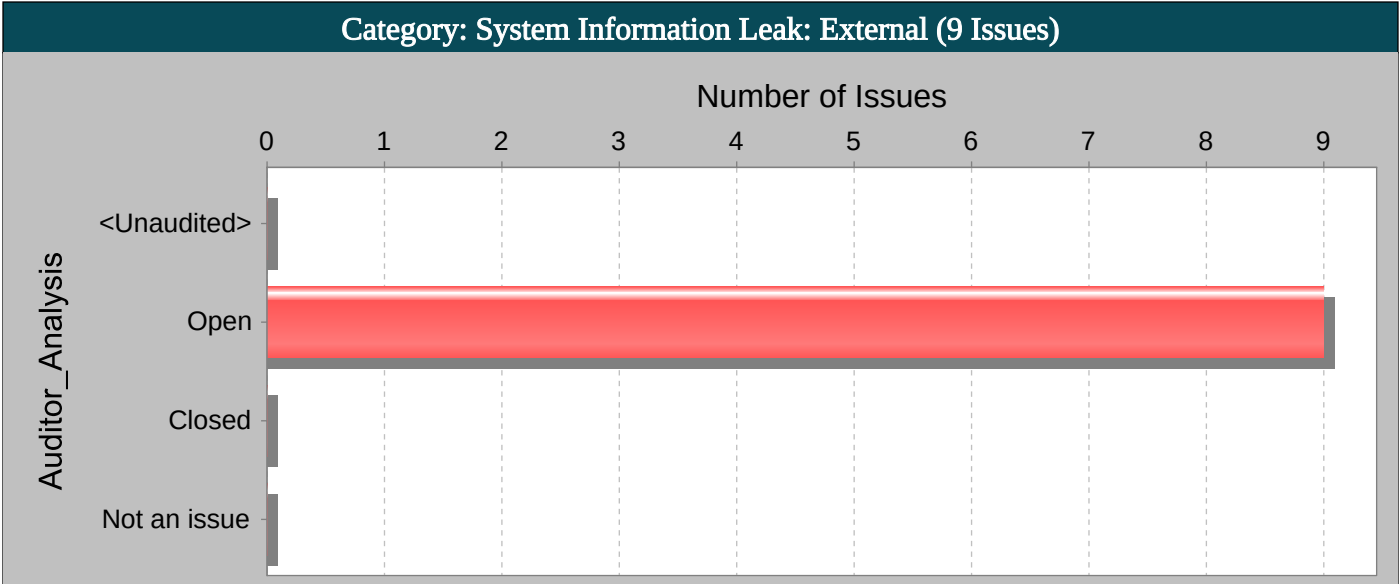
Sink: test_auth_recovery.py:223 FunctionCall: __setitem__()

```

221         json={
222             "user_id": user_id,
223             "password": password,
224             "confirm_password": password,
225             "role": "relationship_manager",

```

Auditor_Analysis: Open			
mcp/docker-compose.yml, line 7 (Password Management: Hardcoded Password)			
Fortify Priority:	High	Folder	High
Kingdom:	Security Features		
Abstract:	Hardcoded passwords can compromise system security in a way that is difficult to remedy.		
Sink:	docker-compose.yml:7 ConfigPair()		
5	POSTGRES_DB: credit_dossier_mcp		
6	POSTGRES_USER: postgres		
7	POSTGRES_PASSWORD: local_credit_dossier		
8	ports:		
9	- "5432:5432"		
Auditor_Analysis: Open			



Abstract:
Revealing system data or debugging information could enable an adversary to use system information to plan an attack.

Explanation:
An external information leak occurs when system data or debug information leaves the program to a remote machine via a socket or network connection. External leaks can help an attacker by revealing specific data about operating systems, full pathnames, the existence of usernames, or locations of configuration files, and are more serious than internal information leaks, which are more difficult for an attacker to access.

Example 1: The following code leaks Exception information into a text area within a web page:

```
...
dirReader.readEntries(function(results){
...
}, function(error){
$("#myTextArea").val("There was a problem: " + error);
});
...
```

This information can be exposed to a remote user. In some cases, the error message provides the attacker with the precise type of attack to which the system is vulnerable. For example, a database error message can reveal that the application is vulnerable to a SQL injection attack. Other error messages can reveal more oblique clues about the system. In Example 1, the leaked information could imply information about the type of operating system, the applications installed on the system, and the amount of care that the administrators have put into configuring the program.

Recommendations:
Write error messages with security in mind. In production environments, turn off detailed error information in favor of brief messages. Restrict the generation and storage of detailed output that can help administrators and programmers diagnose problems. Debug traces can sometimes appear in non-obvious places (embedded in comments in the HTML for an error page, for example).
Even brief error messages that do not reveal stack traces or database dumps can potentially aid an attacker. For example, an "Access Denied" message can reveal that a file or user exists on the system. Because of this, never send information to a resource directly outside the program.

Tips:
1. Do not rely on wrapper scripts, corporate IT policy, or quick-thinking system administrators to prevent system information leaks. Write software that is secure on its own.
2. This category of vulnerability does not apply to all types of programs. For example, if your application executes on a client machine where system information is already available to an attacker, or if you print system information only to a trusted log file, you can use Audit Guide to filter out this category from your scan results.

frontend/src/routes/deals.\$dealId.tsx, line 95 (System Information Leak: External)			
Fortify Priority:	Medium	Folder	Medium
Kingdom:	Encapsulation		
Abstract:	Revealing system data or debugging information could enable an adversary to use system information to plan an attack.		

Source: deals.\$dealId.tsx:73 lambda(0)

```

71      .get(dealId)
72      .then(setDeal)
73      .catch((err) => setError(err.message))
74      .finally(() => setLoading(false));
75    },

```

Sink: deals.\$dealId.tsx:95 Assignment to textContent()

```

93    if (error || !deal) {
94      return (
95        <main className="mx-auto max-w-[1400px] px-3 py-5 sm:px-6 sm:py-8">
96          <p className="text-sm">
97            {error || "Deal not found."}{ " "}

```

Auditor_Analysis: Open

frontend/src/routes/deals.\$dealId.tsx, line 2942 (System Information Leak: External)

Fortify Priority: Medium **Folder** Medium

Kingdom: Encapsulation

Abstract: Revealing system data or debugging information could enable an adversary to use system information to plan an attack.

Source: deals.\$dealId.tsx:73 lambda(0)

```

71      .get(dealId)
72      .then(setDeal)
73      .catch((err) => setError(err.message))
74      .finally(() => setLoading(false));
75    },

```

Sink: deals.\$dealId.tsx:2942 Assignment to textContent()

```

2940      Extract Theme
2941    </label>
2942    <label className="btn-primary inline-flex cursor-pointer items-center
gap-2">
2943      {extractingTheme ? (
2944        <Loader2 className="h-4 w-4 animate-spin" />

```

Auditor_Analysis: Open

frontend/src/routes/login.tsx, line 172 (System Information Leak: External)

Fortify Priority: Medium **Folder** Medium

Kingdom: Encapsulation

Abstract: Revealing system data or debugging information could enable an adversary to use system information to plan an attack.

Source: login.tsx:46 lambda(0)

```

44      );
45    })
46    .catch((requestError) =>
47      setError(
48        requestError instanceof Error ? requestError.message : "Unable to load
password policy.",

```

Sink: login.tsx:172 Assignment to textContent()

```

170
171    return (
172      <main className="relative flex min-h-screen items-center justify-center overflow-
hidden bg-slate-950 px-4 py-10">
173        <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_left,rgba(37,99,235,0.25),transparent_36%),radial-
gradient(circle_at_bottom_right,rgba(14,116,144,0.18),transparent_34%)]" />
174        <div className="relative grid w-full max-w-4xl overflow-hidden rounded-2xl
border border-white/10 bg-white shadow-2xl md:grid-cols-[1.05fr_1fr]">

```

Auditor_Analysis: Open

frontend/src/routes/profile.tsx, line 134 (System Information Leak: External)

Fortify Priority: Medium **Folder** Medium

Kingdom:	Encapsulation
Abstract:	Revealing system data or debugging information could enable an adversary to use system information to plan an attack.
Source:	profile.tsx:53 lambda(0)
51	);
52	})
53	.catch((requestError) =>
54	setQuestionError(
55	requestError instanceof Error
Sink:	profile.tsx:134 Assignment to textContent()
132	
133	return (
134	<main className="mx-auto max-w-3xl px-4 py-8 sm:px-6">
135	<div className="doc-card overflow-hidden">
136	<div className="flex items-center gap-4 border-b bg-primary/5 p-6">
Auditor_Analysis:	Open

frontend/src/routes/deals.\$dealId.tsx, line 1209 (System Information Leak: External)

Fortify Priority:	Medium	Folder	Medium
Kingdom:	Encapsulation		
Abstract:	Revealing system data or debugging information could enable an adversary to use system information to plan an attack.		
Source:	deals.\$dealId.tsx:73 lambda(0)		
71	.get(dealId)		
72	.then(setDeal)		
73	.catch((err) => setError(err.message))		
74	.finally(() => setLoading(false));		
75	},		
Sink:	deals.\$dealId.tsx:1209 Assignment to textContent()		
1207	<div className="flex items-center gap-1">		
1208	{hasTemplate && (		
1209	<>		
1210	<button		
1211	onClick={() => setShowTemplatePreview(!showTemplatePreview)}		
Auditor_Analysis:	Open		

frontend/src/routes/deals.\$dealId.tsx, line 3415 (System Information Leak: External)

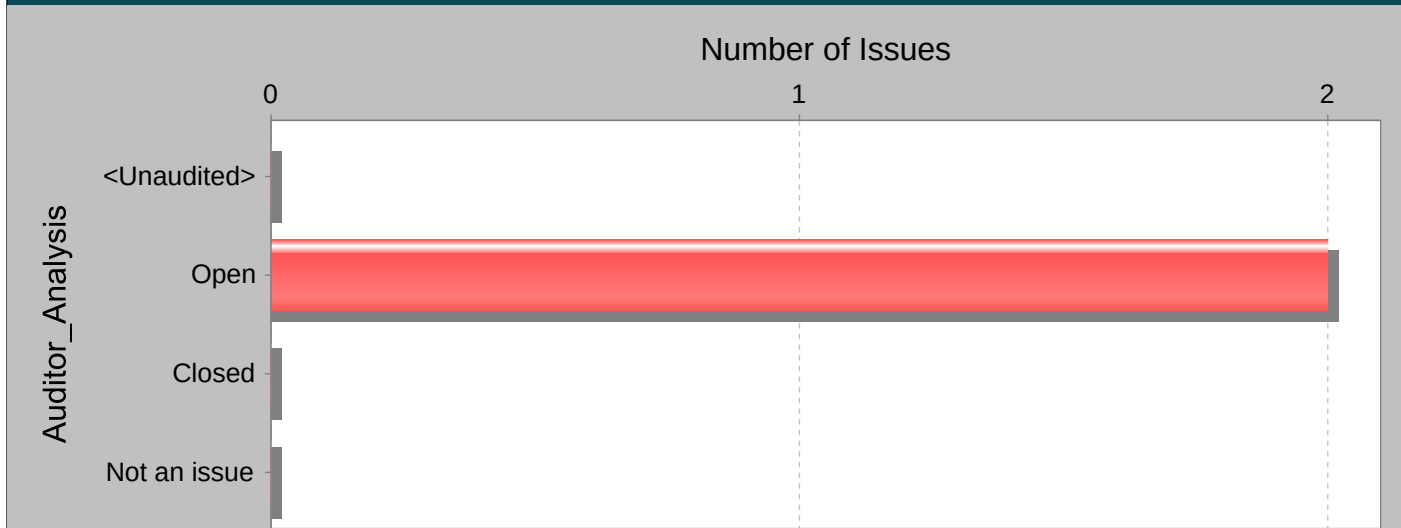
Fortify Priority:	Medium	Folder	Medium
Kingdom:	Encapsulation		
Abstract:	Revealing system data or debugging information could enable an adversary to use system information to plan an attack.		
Source:	deals.\$dealId.tsx:73 lambda(0)		
71	.get(dealId)		
72	.then(setDeal)		
73	.catch((err) => setError(err.message))		
74	.finally(() => setLoading(false));		
75	},		
Sink:	deals.\$dealId.tsx:3415 Assignment to textContent()		
3413			
3414	return (		
3415	<li		
3416	key={lf.id}		
3417	className="flex items-center justify-between px-4 py-2.5 hover:bg-surface/50 transition-colors group"		
Auditor_Analysis:	Open		

frontend/src/routes/login.tsx, line 193 (System Information Leak: External)

Fortify Priority:	Medium	Folder	Medium
--------------------------	--------	---------------	--------

Kingdom:	Encapsulation
Abstract:	Revealing system data or debugging information could enable an adversary to use system information to plan an attack.
Source:	login.tsx:46 lambda(0) <pre> 44); 45 }) 46 .catch((requestError) => 47 setError(48 requestError instanceof Error ? requestError.message : "Unable to load password policy.", </pre>
Sink:	login.tsx:193 Assignment to textContent() <pre> 191 </section> 192 193 <section className="p-6 sm:p-10"> 194 <div className="mb-7 md:hidden"> 195 <div className="flex items-center gap-2 font-semibold text-primary"> </pre>
Auditor_Analysis:	Open
frontend/src/routes/index.tsx, line 85 (System Information Leak: External)	
Fortify Priority:	Medium
Folder	Medium
Kingdom:	Encapsulation
Abstract:	Revealing system data or debugging information could enable an adversary to use system information to plan an attack.
Source:	index.tsx:53 lambda(0) <pre> 51 api.deals.list({ status: statusParam, search: q undefined }) 52 .then(setDeals) 53 .catch(err => setError(err.message)) 54 .finally(() => setLoading(false)); 55 } </pre>
Sink:	index.tsx:85 Assignment to textContent() <pre> 83 84 return (85 <main className="mx-auto max-w-[1400px] px-3 py-5 sm:px-6 sm:py-8"> 86 <div className="mb-6 flex flex-wrap items-end justify-between gap-3"> 87 <div> </pre>
Auditor_Analysis:	Open
frontend/src/routes/deals.\$dealId.tsx, line 2743 (System Information Leak: External)	
Fortify Priority:	Medium
Folder	Medium
Kingdom:	Encapsulation
Abstract:	Revealing system data or debugging information could enable an adversary to use system information to plan an attack.
Source:	deals.\$dealId.tsx:73 lambda(0) <pre> 71 .get(dealId) 72 .then(setDeal) 73 .catch((err) => setError(err.message)) 74 .finally(() => setLoading(false)); 75 }, </pre>
Sink:	deals.\$dealId.tsx:2743 Assignment to textContent() <pre> 2741 <ul className="divide-y"> 2742 {deal.versions.map((v) => (2743 <li key={v.id} className="p-4 text-sm"> 2744 <div className="flex items-start justify-between gap-4"> 2745 <div> </pre>
Auditor_Analysis:	Open

Category: Cross-Site Request Forgery (2 Issues)

**Abstract:**

The HTTP request at deals.\$dealId.tsx line 2879 must contain a user-specific secret to prevent an attacker from making unauthorized requests.

Explanation:

A cross-site request forgery (CSRF) vulnerability occurs when:

1. A web application uses session cookies.
2. The application acts on an HTTP request without verifying that the request was made with the user's consent.

A nonce is a cryptographic random value that is sent with a message to prevent replay attacks. If the request does not contain a nonce that proves its provenance, the code that handles the request is vulnerable to a CSRF attack (unless it does not change the state of the application). This means a web application that uses session cookies has to take special precautions to ensure that an attacker can't trick users into submitting bogus requests. Imagine a web application that allows administrators to create new accounts as follows:

```
var req = new XMLHttpRequest();
req.open("POST", "/new_user", true);
body = addToPost(body, new_username);
body = addToPost(body, new_passwd);
req.send(body);
```

An attacker might set up a malicious web site that contains the following code.

```
var req = new XMLHttpRequest();
req.open("POST", "http://www.example.com/new_user", true);
body = addToPost(body, "attacker");
body = addToPost(body, "haha");
req.send(body);
```

If an administrator for example.com visits the malicious page while she has an active session on the site, she will unwittingly create an account for the attacker. This is a CSRF attack. It is possible because the application does not have a way to determine the provenance of the request. Any request could be a legitimate action chosen by the user or a faked action set up by an attacker. The attacker does not get to see the Web page that the bogus request generates, so the attack technique is only useful for requests that alter the state of the application.

Applications that pass the session identifier in the URL rather than as a cookie do not have CSRF problems because there is no way for the attacker to access the session identifier and include it as part of the bogus request.

Recommendations:

Applications that use session cookies must include some piece of information in every form post that the back-end code can use to validate the provenance of the request. One way to do that is to include a random request identifier or nonce, as follows:

```
RequestBuilder rb = new RequestBuilder(RequestBuilder.POST, "/new_user");
body = addToPost(body, new_username);
body = addToPost(body, new_passwd);
body = addToPost(body, request_id);
```

```
rb.sendRequest(body, new NewAccountCallback(callback));
```

Then the back-end logic can validate the request identifier before processing the rest of the form data. When possible, the request identifier should be unique to each server request rather than shared across every request for a particular session. As with session identifiers, the harder it is for an attacker to guess the request identifier, the harder it is to conduct a successful CSRF attack. The token should not be easily guessed and it should be protected in the same way that session tokens are protected, such as using SSLv3.

Additional mitigation techniques include:

Framework protection: Most modern web application frameworks embed CSRF protection and they will automatically include and verify CSRF tokens.

Use a Challenge-Response control: Forcing the customer to respond to a challenge sent by the server is a strong defense against CSRF. Some of the challenges that can be used for this purpose are: CAPTCHAs, password re-authentication and one-time tokens.

Check HTTP Referer/Origin headers: An attacker won't be able to spoof these headers while performing a CSRF attack. This makes these headers a useful method to prevent CSRF attacks.

Double-submit Session Cookie: Sending the session ID Cookie as a hidden form value in addition to the actual session ID Cookie is a good protection against CSRF attacks. The server will check both values and make sure they are identical before processing the rest of the form data. If an attacker submits a form in behalf of a user, he won't be able to modify the session ID cookie value as per the same-origin-policy.

Limit Session Lifetime: When accessing protected resources using a CSRF attack, the attack will only be valid as long as the session ID sent as part of the attack is still valid on the server. Limiting the Session lifetime will reduce the probability of a successful attack.

The techniques described here can be defeated with XSS attacks. Effective CSRF mitigation includes XSS mitigation techniques.

Tips:

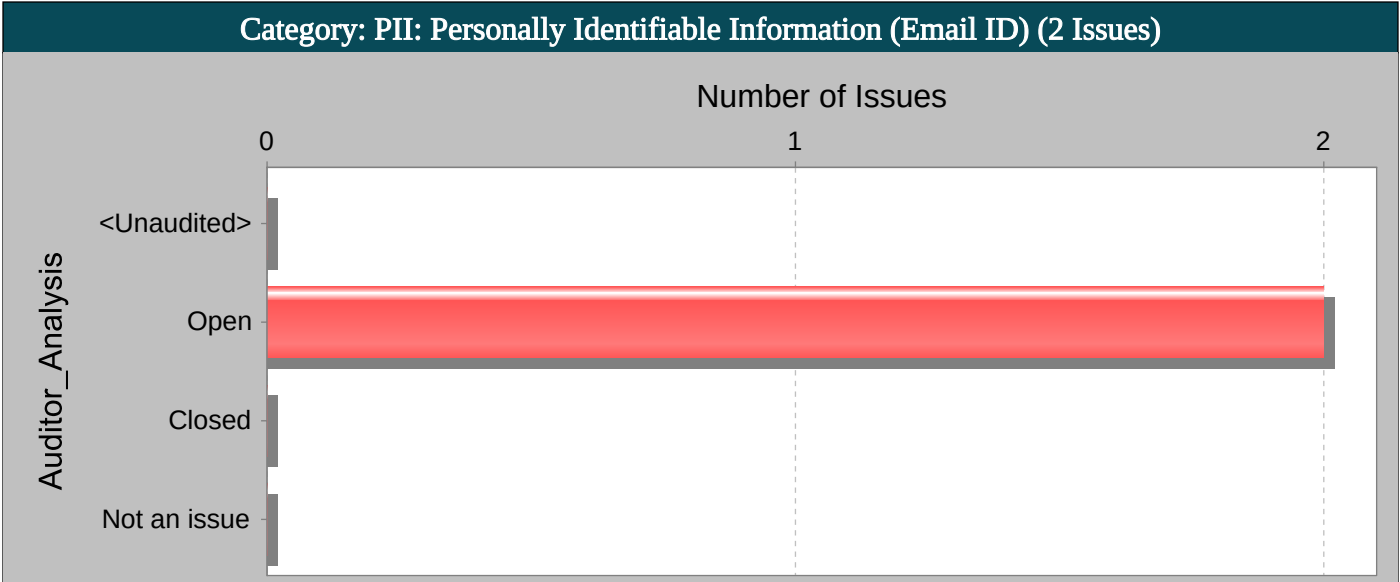
1. Fortify Static Code Analyzer flags all HTML forms and all XMLHttpRequest objects that might perform either a GET or POST operation. The auditor must determine if each form is valuable to an attacker as a CSRF target and whether or not an appropriate mitigation technique is in place.

frontend/src/routes/deals.\$dealId.tsx, line 2914 (Cross-Site Request Forgery)

Fortify Priority:	Low	Folder	Low
Kingdom:	Encapsulation		
Abstract:	The HTTP request at deals.\$dealId.tsx line 2914 must contain a user-specific secret to prevent an attacker from making unauthorized requests.		
Sink:	deals.\$dealId.tsx:2914 AssignmentStatement()		
2912			
2913	const res = await fetch(`/api/deals/\${deal.id}`, {		
2914	method: "PATCH",		
2915	headers: { "Content-Type": "application/json" },		
2916	body: JSON.stringify(payload),		
Auditor_Analysis:	Open		

frontend/src/routes/deals.\$dealId.tsx, line 2879 (Cross-Site Request Forgery)

Fortify Priority:	Low	Folder	Low
Kingdom:	Encapsulation		
Abstract:	The HTTP request at deals.\$dealId.tsx line 2879 must contain a user-specific secret to prevent an attacker from making unauthorized requests.		
Sink:	deals.\$dealId.tsx:2879 AssignmentStatement()		
2877	formData.append("file", file);		
2878	const res = await fetch(`/api/deals/\${deal.id}/theme/extract`, {		
2879	method: "POST",		
2880	body: formData,		
2881	});		
Auditor_Analysis:	Open		



Abstract:

Personally Identifiable Information (PII) refers to any data that can be used to identify an individual, either directly or indirectly. This includes names, social security numbers, email addresses, phone numbers, and other data that can uniquely distinguish a person. The protection of PII is critical in today’s digital landscape, where data breaches and unauthorized access can lead to significant privacy violations and identity theft.

Explanation:

Personally Identifiable Information (PII) encompasses any data that can identify an individual, such as names, addresses, and social security numbers. Protecting PII is crucial due to the risks of identity theft and fraud, which can have serious consequences for individuals and organizations alike. Regulatory frameworks like the GDPR and CCPA mandate strict safeguards for handling PII, emphasizing the need for explicit consent and transparency. Implementing best practices, such as data minimization, access controls, and encryption, is essential for ensuring compliance and maintaining trust in today's digital landscape.

Recommendations:

Organizations should establish comprehensive data protection policies that emphasize minimizing the collection of Personally Identifiable Information (PII) and obtaining explicit user consent. Regular employee training on best practices for data handling is crucial to mitigate the risk of breaches. Implementing strong access controls and encryption is essential to protect sensitive information both during transmission and storage. Furthermore, conducting periodic audits can help identify vulnerabilities and ensure ongoing compliance with regulations, fostering trust with customers.

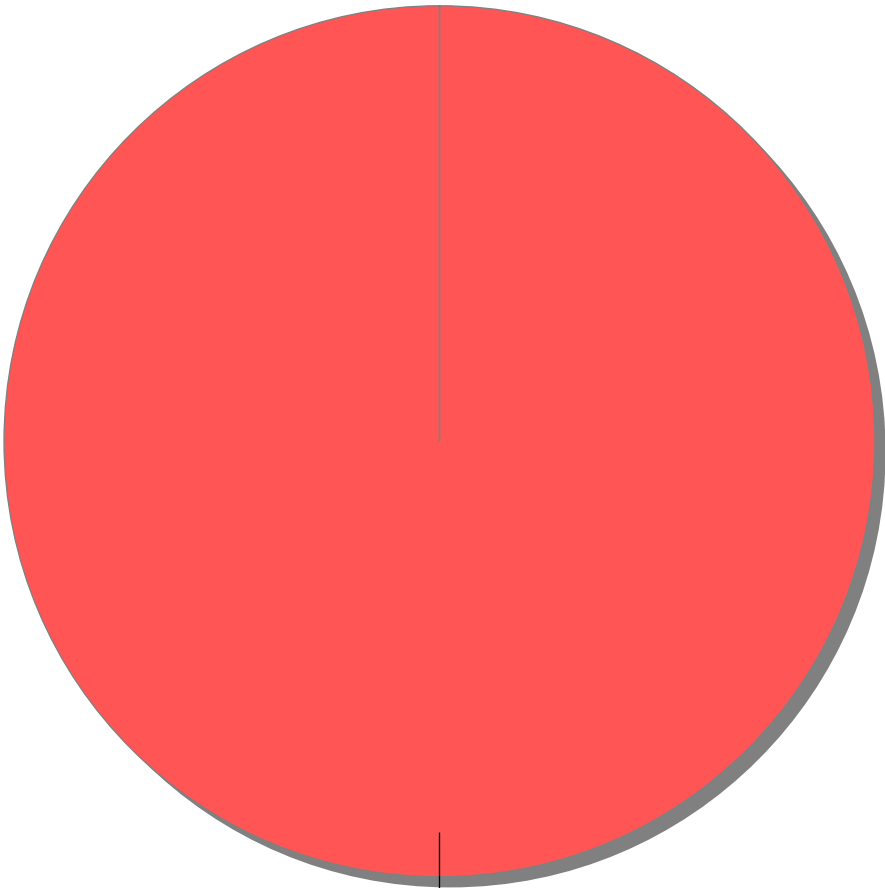
backend/tests/test_security_controls.py, line 34 (PII: Personally Identifiable Information (Email ID))			
Fortify Priority:	Low	Folder	Low
Kingdom:	Environment		
Abstract:	Personally Identifiable Information (PII) refers to any data that can be used to identify an individual, either directly or indirectly. This includes names, social security numbers, email addresses, phone numbers, and other data that can uniquely distinguish a person. The protection of PII is critical in today’s digital landscape, where data breaches and unauthorized access can lead to significant privacy violations and identity theft.		
Sink:	test_security_controls.py:34		
32	with patch.object(security, "_cached_key", None):		
33	masked = security.mask_sensitive_text(source)		
34	self.assertNotIn("alice@example.com", masked)		
35	self.assertNotIn("ABCDE1234F", masked)		
36	self.assertNotIn("12345678901", masked)		
Auditor_Analysis:	Open		
backend/tests/test_security_controls.py, line 30 (PII: Personally Identifiable Information (Email ID))			
Fortify Priority:	Low	Folder	Low
Kingdom:	Environment		

Abstract:	Personally Identifiable Information (PII) refers to any data that can be used to identify an individual, either directly or indirectly. This includes names, social security numbers, email addresses, phone numbers, and other data that can uniquely distinguish a person. The protection of PII is critical in today’s digital landscape, where data breaches and unauthorized access can lead to significant privacy violations and identity theft.
Sink:	test_security_controls.py:30
28	
29	def test_report_masking_removes_recognized_identifiers(self):
30	source = "Email alice@example.com, PAN ABCDE1234F, account number 12345678901"
31	with patch.object(security.settings, "REPORT_TOKENIZATION_KEY", self.key):
32	with patch.object(security, "_cached_key", None):
Auditor_Analysis:	Open

Issue Count by Category	
Issues by Category	
Privacy Violation	51
Password Management: Hardcoded Password	24
System Information Leak: External	9
Cross-Site Request Forgery	2
PII: Personally Identifiable Information (Email ID)	2

Issue Breakdown by Analysis

Issues by Analysis



● <none>